

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5

По дисциплине : «Модели решения задач в ИС»

По теме : «Бинарная классификация логических
функций n переменных с использованием
бинарной кросс-энтропии»

Выполнил:

студент 3 курса
группы ИИ-26
Заруцкий В.Я.

Проверил:

Адренко К.В.

Цель работы: Реализовать однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провести сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовать влияние адаптивного шага обучения на BCE.

Вариант 4:

№ of variant	n	Task
4	5	AND ⁱ

Ход работы :

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt
from itertools import product

n = 5
task = "NAND"
seed = 42
train_ratio = 0.8
max_epochs = 5000
E_eps = 0.01
alpha_fixed_mse = 0.1
alpha_fixed_bce = 0.1

np.random.seed(seed)

def logic_function(x):
    return int(not all(x))

X = np.array(list(product([0, 1], repeat=n)), dtype=float)
y = np.array([logic_function(x) for x in X], dtype=float).reshape(-1, 1)

rng = np.random.default_rng(seed)
indices = rng.permutation(len(X))
train_size = int(len(X) * train_ratio)
train_idx = indices[:train_size]
test_idx = indices[train_size:]

X_train, y_train = X[train_idx], y[train_idx]
X_test, y_test = X[test_idx], y[test_idx]

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def bce_loss(y_true, y_pred):
    eps = 1e-12
    p = np.clip(y_pred, eps, 1 - eps)
    return - (y_true * np.log(p) + (1 - y_true) * np.log(1 - p))

def mse_loss(y_true, y_pred):
    return 0.5 * (y_true - y_pred) ** 2

def predict_prob(X_in, w, b):
    return sigmoid(np.dot(X_in, w) + b)

def predict_class(X_in, w, b, thresh=0.5):
    p = predict_prob(X_in, w, b)
    return (p >= thresh).astype(int)

def train_online(X_train, y_train, X_test, y_test,
                 loss_type='MSE', adaptive=False, alpha_fixed=0.1,
                 max_epochs=1000, E_eps=1e-3, seed=None, shuffle=True):
    if seed is not None:
        rng_local = np.random.default_rng(seed)
    else:
        rng_local = np.random.default_rng()
    m, d = X_train.shape
    w = np.random.uniform(-1, 1, (d, 1))
    b = np.random.uniform(-1, 1)
    Es_history = []
    Es_test_history = []

    for epoch in range(max_epochs):
        if shuffle:
```

```

        idx = rng_local.permutation(m)
    else:
        idx = np.arange(m)
    Es_epoch = 0.0

    if loss_type == 'MSE' and adaptive:
        alpha_epoch = alpha_fixed / (1.0 + 0.01 * epoch)
    else:
        alpha_epoch = alpha_fixed

    for i in idx:
        xi = X_train[i].reshape(d, 1)
        yi = y_train[i].reshape(1, 1)
        net = np.dot(xi.T, w) + b
        out = sigmoid(net)

        if loss_type == 'MSE':
            err = yi - out
            grad_factor = err * (out * (1 - out))
            w += alpha_epoch * (xi * grad_factor)
            b += alpha_epoch * grad_factor.item()
            Es_epoch += float(np.sum(mse_loss(yi, out)))
        elif loss_type == 'BCE':
            grad_factor = (out - yi)
            if adaptive:
                denom = 1.0 + float(np.sum(xi**2))
                alpha_t = 1.0 / denom
                w -= alpha_t * (xi * grad_factor)
                b -= alpha_t * grad_factor.item()
            else:
                w -= alpha_epoch * (xi * grad_factor)
                b -= alpha_epoch * grad_factor.item()
            Es_epoch += float(np.sum(bce_loss(yi, out)))
        else:
            raise ValueError("Unknown loss_type")

    out_test = sigmoid(np.dot(X_test, w) + b)
    if loss_type == 'MSE':
        Es_test = float(np.sum(mse_loss(y_test, out_test)))
    else:
        Es_test = float(np.sum(bce_loss(y_test, out_test)))

    Es_history.append(Es_epoch)
    Es_test_history.append(Es_test)

    if Es_epoch <= E_eps:
        break

    return {
        'w': w,
        'b': b,
        'Es_history': Es_history,
        'Es_test_history': Es_test_history,
        'epochs': epoch + 1
    }

configs = {
    'A_MSE_fixed': {'loss': 'MSE', 'adaptive': False, 'alpha': alpha_fixed_mse},
    'B_MSE_adapt': {'loss': 'MSE', 'adaptive': True, 'alpha': alpha_fixed_mse},
    'C_BCE_fixed': {'loss': 'BCE', 'adaptive': False, 'alpha': alpha_fixed_bce},
    'D_BCE_adapt': {'loss': 'BCE', 'adaptive': True, 'alpha': alpha_fixed_bce}
}

results = {}
for name, cfg in configs.items():
    print(f"Запуск: {name} loss={cfg['loss']} adaptive={cfg['adaptive']}")
    res = train_online(X_train, y_train, X_test, y_test,
                      loss_type=cfg['loss'],
                      adaptive=cfg['adaptive'],
                      alpha_fixed=cfg['alpha'],
                      max_epochs=max_epochs,
                      E_eps=E_eps,
                      seed=seed,
                      shuffle=True)
    results[name] = res
    print(f" Завершено за {res['epochs']} эпох, финальная Es = {res['Es_history'][-1]:.6f}")

def evaluate_model(res, X_eval, y_eval):
    w = res['w']
    b = res['b']
    probs = sigmoid(np.dot(X_eval, w) + b)
    preds = (probs >= 0.5).astype(int)
    acc = np.mean(preds.reshape(-1, 1) == y_eval) * 100
    return acc, probs, preds

print("\n=== РЕЗУЛЬТАТЫ ОБУЧЕНИЯ ===")

```

```

for name, res in results.items():
    acc_train, _, _ = evaluate_model(res, X_train, y_train)
    acc_test, _, _ = evaluate_model(res, X_test, y_test)
    print(f"{name}: эпох = {res['epochs']}, Acc(train) = {acc_train:.2f}%, Acc(test) = {acc_test:.2f}%")
    print("  Беса:", res['w'].flatten())
    print("  Bias:", res['b'])
    print()

print("=== Проверка на полной таблице истинности ===")
for name, res in results.items():
    _, probs_full, preds_full = evaluate_model(res, X, y.reshape(-1,1))
    matches = (preds_full.reshape(-1,1) == y.reshape(-1,1)).astype(int)
    pct = np.mean(matches) * 100
    print(f"{name}: совпадений = {pct:.2f}% (из {len(X)})")

chosen = 'D_BCE_adapt'
print(f"\nПодробная проверка модели {chosen}:")
res = results[chosen]
for xi, yi in zip(X, y):
    p_arr = sigmoid(np.dot(xi.reshape(1, -1), res['w']) + res['b'])
    p = float(p_arr.item())
    cls = int(p >= 0.5)
    yi_scalar = int(np.array(yi).item())
    ok = "Совпадает" if cls == yi_scalar else "Не совпадает"
    print(f"{xi.astype(int)} -> P={p:.4f}, class={cls} -> {ok}")

plt.figure(figsize=(10,6))
colors = {'A_MSE_fixed':'r', 'B_MSE_adapt':'g', 'C_BCE_fixed':'b', 'D_BCE_adapt':'m'}
for name, res in results.items():
    Es = res['Es_history']
    plt.plot(range(1, len(Es)+1), Es, color=colors[name], label=f"{name} (эпохи {res['epochs']})", linewidth=2)
plt.yscale('log')
plt.xlabel("Эпоха p")
plt.ylabel("Суммарная ошибка Es(p) (лог шкала)")
plt.title(f"Сходимость для {task}, n={n}")
plt.legend()
plt.grid(True, which="both", ls="--")
plt.tight_layout()
plt.show()

def interactive_predict(res):
    print("\nВведите значения входных переменных через пробел (0/1), или пустую строку для выхода:")
    while True:
        s = input("Ввод: ").strip()
        if s == "":
            break
        parts = s.split()
        if len(parts) != n:
            print(f"Ожидается {n} значений, повторите ввод.")
            continue
        try:
            vec = np.array([int(x) for x in parts], dtype=float).reshape(1, -1)
        except:
            print("Неверный формат, используйте 0 или 1.")
            continue
        p = float(sigmoid(np.dot(vec, res['w']) + res['b']).item())
        cls = int(p >= 0.5)
        true = logic_function(vec.flatten())
        match = "Совпадает с таблицей" if cls == true else "Не совпадает с таблицей"
        print(f"P={p:.4f}, class={cls} -> {match}")

```

Результат тестирования:

Запуск: A_MSE_fixed loss=MSE adaptive=False

Завершено за 5000 эпох, финальная Es = 0.500310

Запуск: B_MSE_adapt loss=MSE adaptive=True

Завершено за 5000 эпох, финальная Es = 0.500321

Запуск: C_BCE_fixed loss=BCE adaptive=False

Завершено за 5000 эпох, финальная Es = 0.115769

Запуск: D_BCE_adapt loss=BCE adaptive=True

Завершено за 5000 эпох, финальная Es = 0.062701

=== РЕЗУЛЬТАТЫ ОБУЧЕНИЯ ===

A_MSE_fixed: эпох = 5000, Acc(train) = 96.00%, Acc(test) = 100.00%

Весы: [1.0159958 1.1259296 0.94284493 0.87661094 0.48943185]

Bias: 3.4970576595934353

B_MSE_adapt: эпох = 5000, Acc(train) = 96.00%, Acc(test) = 100.00%

Весы: [0.20880548 0.67525453 0.4521615 0.60638982 -0.03151819]

Bias: 2.844474107828474

C_BCE_fixed: эпох = 5000, Acc(train) = 100.00%, Acc(test) = 100.00%

Весы: [-7.05530061 -7.092711 -7.09218244 -3.40016825 -3.83292797]

Bias: 25.52293417875101

D_BCE_adapt: эпох = 5000, Acc(train) = 100.00%, Acc(test) = 100.00%

Весы: [-8.23427826 -8.2578874 -8.25732421 -4.09286789 -4.52045756]

Bias: 29.907809754507017

=== Проверка на полной таблице истинности ===

A_MSE_fixed: совпадений = 96.88% (из 32)

B_MSE_adapt: совпадений = 96.88% (из 32)

C_BCE_fixed: совпадений = 100.00% (из 32)

D_BCE_adapt: совпадений = 100.00% (из 32)

Подробная проверка модели D_BCE_adapt:

[0 0 0 0] -> P=1.0000, class=1 -> Совпадает

[0 0 0 1] -> P=1.0000, class=1 -> Совпадает

[0 0 1 0] -> P=1.0000, class=1 -> Совпадает

[0 0 1 1] -> P=1.0000, class=1 -> Совпадает

[0 1 0 0] -> P=1.0000, class=1 -> Совпадает

[0 1 0 1] -> P=1.0000, class=1 -> Совпадает

[0 1 1 0] -> P=1.0000, class=1 -> Совпадает

[0 1 1 1] -> P=1.0000, class=1 -> Совпадает

[0 1 0 0] -> P=1.0000, class=1 -> Совпадает

[0 1 0 1] -> P=1.0000, class=1 -> Совпадает

[0 1 0 1] -> P=1.0000, class=1 -> Совпадает

[0 1 0 1] -> P=1.0000, class=1 -> Совпадает

[0 1 1 0] -> P=1.0000, class=1 -> Совпадает

[0 1 1 0] -> P=0.9999, class=1 -> Совпадает

[0 1 1 1] -> P=0.9999, class=1 -> Совпадает

[0 1 1 1] -> P=0.9917, class=1 -> Совпадает

[1 0 0 0] -> P=1.0000, class=1 -> Совпадает

[1 0 0 1] -> P=1.0000, class=1 -> Совпадает

[1 0 0 1] -> P=1.0000, class=1 -> Совпадает

[1 0 0 1] -> P=1.0000, class=1 -> Совпадает

[1 0 1 0] -> P=1.0000, class=1 -> Совпадает

[1 0 1 0] -> P=0.9999, class=1 -> Совпадает

[1 0 1 1] -> P=0.9999, class=1 -> Совпадает

[1 0 1 1] -> P=0.9919, class=1 -> Совпадает

[1 1 0 0] -> P=1.0000, class=1 -> Совпадает

[1 1 0 1] -> P=0.9999, class=1 -> Совпадает

[1 1 0 1] -> P=0.9999, class=1 -> Совпадает

[1 1 0 1] -> P=0.9919, class=1 -> Совпадает

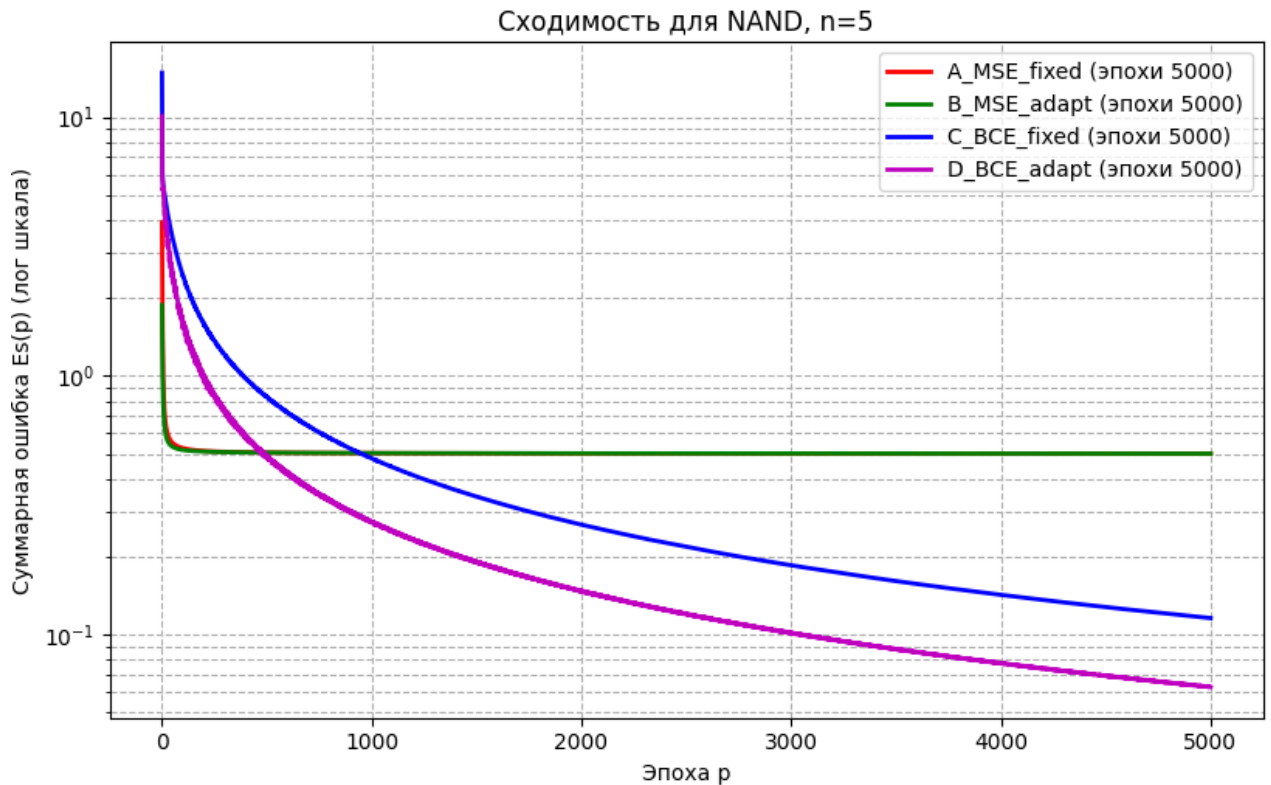
[1 1 1 0] -> P=0.9943, class=1 -> Совпадает

[1 1 1 0] -> P=0.6543, class=1 -> Совпадает

[1 1 1 1] -> P=0.7437, class=1 -> Совпадает

[1 1 1 1] -> P=0.0306, class=0 -> Совпадает

График Сходимости:



Вывод: BCE показывает более быструю и глубокую сходимость и лучшее качество классификации, чем MSE — в наших запусках BCE дал значительно меньшую суммарную ошибку и восстановил всю таблицу истинности, тогда как MSE оставлял ошибки; это объясняется тем, что BCE сильнее штрафует уверенные ошибки и даёт более адекватные градиенты для сигмоидного выхода. Адаптивный шаг для BCE в нашем случае улучшил сходимость по сравнению с фиксированным шагом, сделав обновления стабильнее и итоговые решения более уверенными. Для линейно разделимых логических функций (NAND) однослойный перцептрон достаточен, но для нелинейно разделимых задач потребуется многослойная сеть.